

End-to-End Data Engineering Architecture

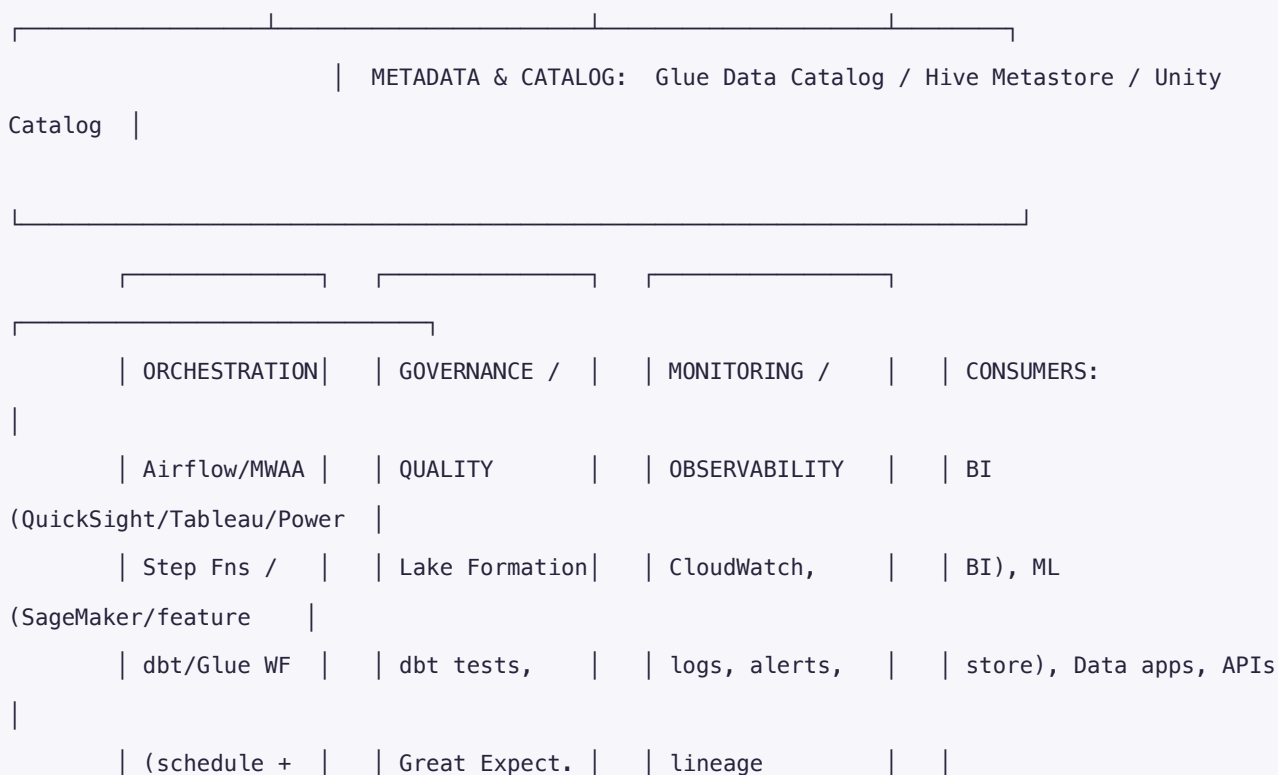
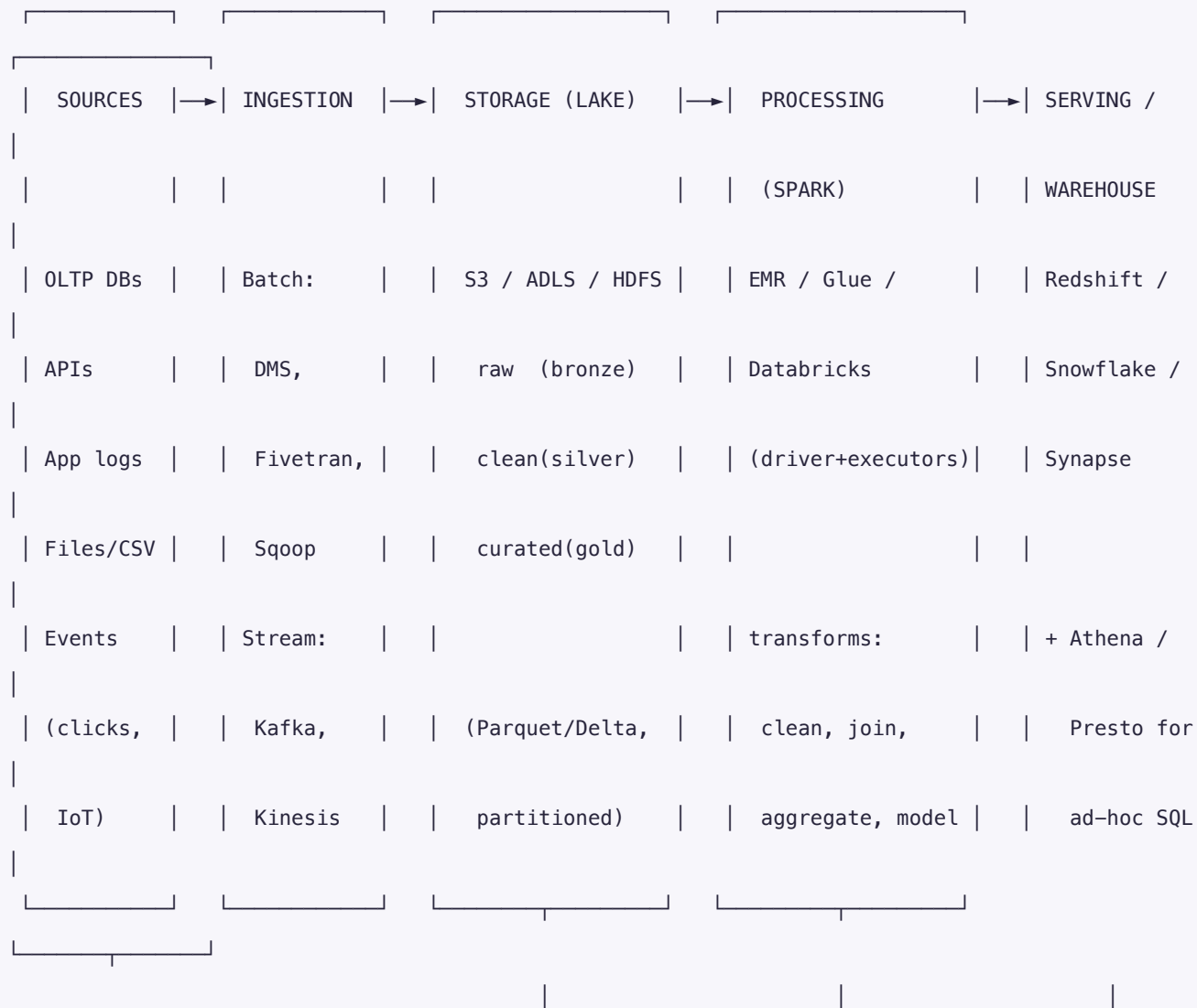
What happens when a data engineer submits a Spark job — all components, with a real scenario

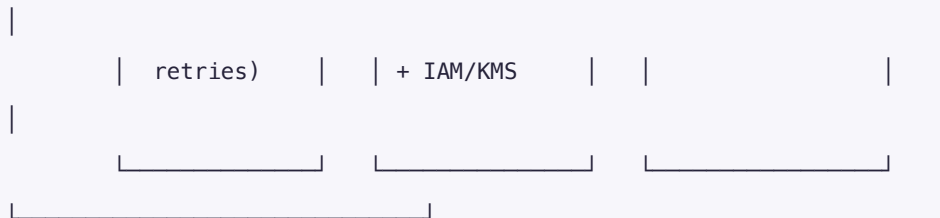
Sources → ingestion → lake → Spark → catalog → warehouse → BI/ML | driver/executors/DAG/stages/shuffle

A complete, detailed walkthrough of the data engineer's world: every component of a modern data platform, how data flows through all of them, and — in depth — what happens inside the cluster when a data engineer submits a Spark job. Built around one real-time scenario: **"ShopFast," an e-commerce company** processing daily orders and clickstream to power analytics and ML.

PART 1 — The complete data platform (every component of the role)

A data engineer owns the pipeline from raw source data to trustworthy, query-ready data. Here is the full landscape, layer by layer.

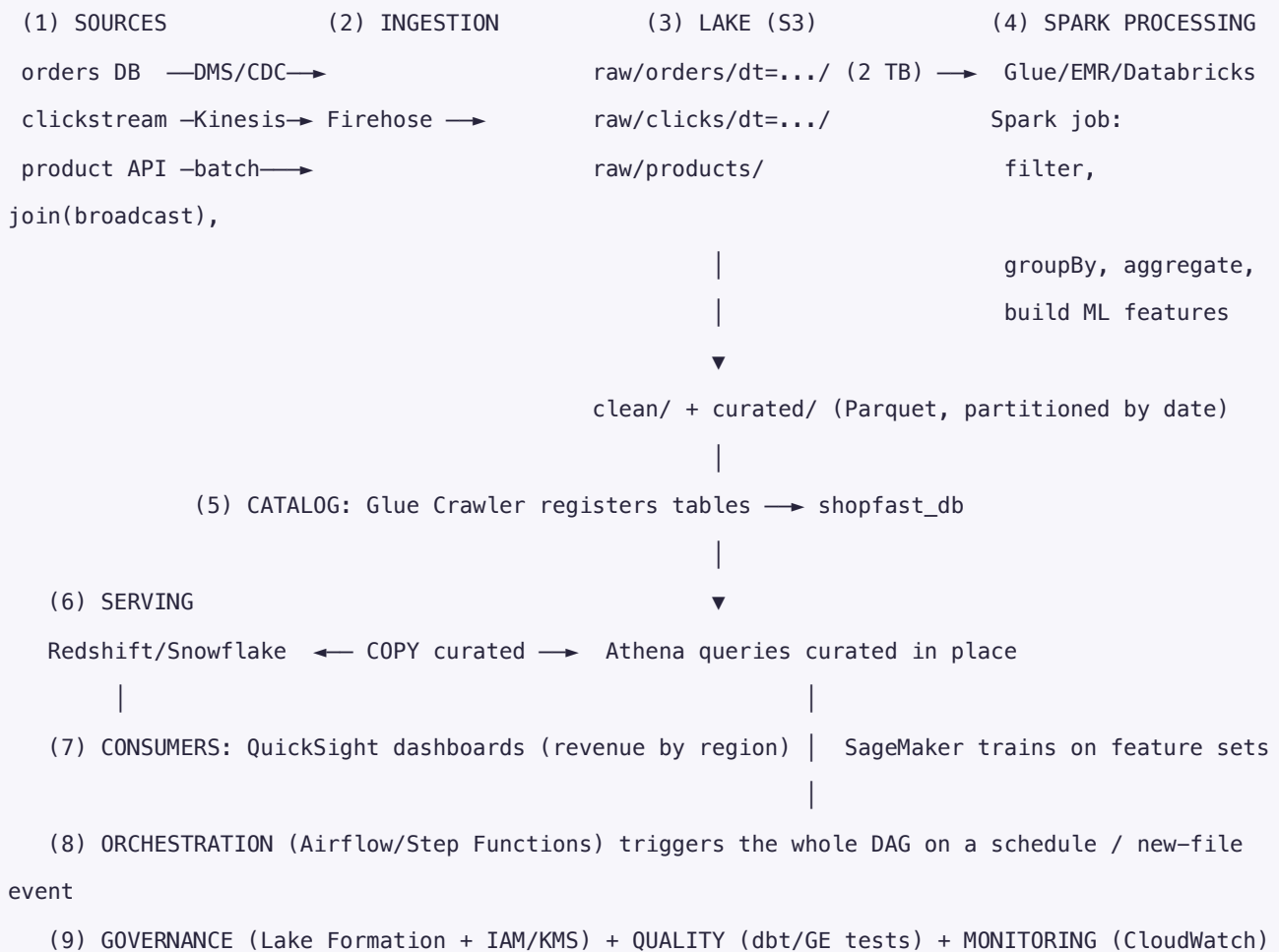




Everything is defined as code (Terraform/CloudFormation) and shipped through CI/CD (Git/Jenkins).

The layers and what each does:

Layer	Purpose	Typical tools
Sources	Where data originates	OLTP DBs (Oracle/Postgres), REST APIs, app/server logs, files, event streams (clicks, IoT)
Ingestion	Move data into the lake	Batch: AWS DMS/CDC , Fivetran, Sqoop. Stream: Kafka, Kinesis, Event Hubs
Storage (data lake)	Cheap, durable landing + curated zones	S3 / ADLS / HDFS ; medallion zones raw → clean → curated ; Parquet/Delta, partitioned
Processing	Clean, join, aggregate, model at scale	Apache Spark on EMR / AWS Glue / Databricks
Catalog/Metadata	Schema so engines can query files	Glue Data Catalog , Hive Metastore, Unity Catalog
Serving / Warehouse	Fast SQL for BI	Redshift / Snowflake / Synapse ; Athena/Presto for ad-hoc
Orchestration	Schedule + sequence + retry pipelines	Airflow/MWAA , Step Functions, Glue Workflows, dbt
Governance / Quality	Access control, PII, data quality, lineage	Lake Formation , IAM/KMS, dbt tests, Great Expectations
Monitoring	Health, latency, cost, failures	CloudWatch , Spark UI, logs, alerts
Consumers	Who uses the data	BI dashboards, ML/feature stores, data apps, APIs
IaC / CI-CD	Reproducible infra + deployments	Terraform/CloudFormation , Git, Jenkins/Azure DevOps



Narration:

- Sources:** orders live in an OLTP database; clickstream is a firehose of events; product data comes from an API.
- Ingestion:** **DMS** captures order changes (CDC); **Kinesis + Firehose** stream clicks to S3 as Parquet; a batch job pulls the product API.
- Lake:** everything lands in the **raw** zone, immutable, partitioned by date.
- Processing (Spark):** the data engineer's **Spark job** cleans and joins the data, computes **revenue per region**, and builds **ML feature tables** — writing Parquet to **curated**. *(Part 3 is exactly what happens when this job is submitted.)*
- Catalog:** a **Glue Crawler** registers the curated tables so SQL engines can see them.
- Serving:** curated data is **COPY**-loaded into **Redshift/Snowflake** for fast BI, and **Athena** queries it in place for ad-hoc.
- Consumers:** **QuickSight** shows revenue by region; **SageMaker** trains the recommender on the feature tables.
- Orchestration:** **Airflow/Step Functions** runs ingest → Spark → crawl → load → quality in order, with retries and alerts, triggered on a schedule or by a new-file event (EventBridge).
- Governance/Quality/Monitoring:** **Lake Formation + IAM/KMS** control access and encryption; **dbt/Great Expectations** tests fail the run on bad data; **CloudWatch** watches health/cost.

PART 3 — Deep dive: what happens when the data engineer submits the Spark job

This is step (4) above, in full detail. The job: **revenue per region over 2 TB of orders**.

3.1 The code

```
from pyspark.sql.functions import col, sum as _sum, broadcast
orders = spark.read.parquet("s3://shopfast/raw/orders/dt=2026-08-10/") # 2 TB
regions = spark.read.parquet("s3://shopfast/dim/regions/") # few MB
result = (orders
    .filter(col("status") == "completed") # narrow – no shuffle
    .join(broadcast(regions), "region_id") # broadcast small table – avoids big shuffle
    .groupBy("region_name") # WIDE – triggers a shuffle
    .agg(_sum("amount").alias("revenue")))
result.write.mode("overwrite").parquet("s3://shopfast/curated/revenue_by_region/") # ACTION
```

3.2 Submitting it

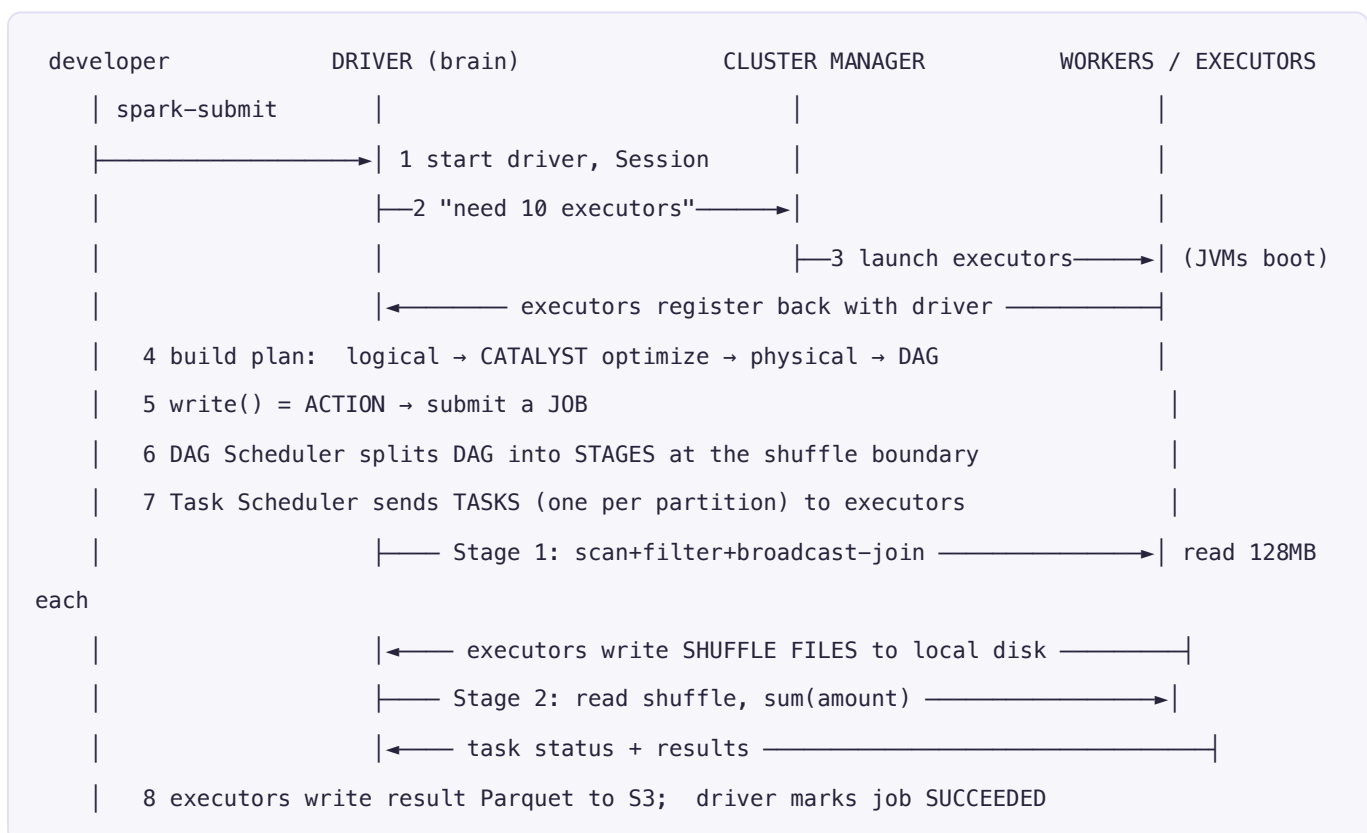
```
spark-submit --master yarn --deploy-mode cluster \
    --num-executors 10 --executor-cores 4 --executor-memory 16g revenue_by_region.py
```

On **Glue/Databricks/EMR** you pick worker size + count instead; they run `spark-submit` for you.

3.3 The components involved

Component	Role
Driver	Runs your code + SparkSession/SparkContext , builds the DAG , schedules and tracks work. The brain.
Cluster Manager (YARN / Kubernetes / Standalone / Glue/Databricks-managed)	Allocates resources; launches executors on worker nodes.
Worker node	A machine in the cluster that hosts executors.
Executor	A JVM process on a worker that runs tasks and holds cached/shuffle data. Here 10×4 cores = 40 parallel tasks .
Partition	~128 MB chunk of data = the unit of parallelism . $2 \text{ TB} \div 128 \text{ MB} \approx$ 16,000 partitions .
Task	Processes one partition on one core.
DAG Scheduler	Splits the job into stages at shuffle boundaries.
Task Scheduler	Sends tasks to executors and handles retries.
Catalyst optimizer / Tungsten	Optimizes the plan; generates efficient JVM code.

3.4 The end-to-end execution flow



Step by step:

1. **Submit.** `spark-submit` asks the **cluster manager** to start the **driver** (cluster mode).
2. **Driver starts** and creates the **SparkSession** (SparkContext inside).

3. **Resource request.** Driver asks for executors; YARN/K8s finds capacity and **launches 10 executors** on worker nodes; they **register** with the driver.
4. **Build the plan (lazy — nothing runs yet).** The driver turns your transformations into a **logical plan**, **Catalyst** optimizes it (pushes the `status` filter into the Parquet scan, prunes unused columns, picks a **BroadcastHashJoin** because `regions` is tiny), and produces a **physical plan** → a **DAG**.
5. **Action fires.** `write()` is an **action**, so the driver submits a **Job**.
6. **DAG → Stages.** The **DAG Scheduler** cuts the job at the shuffle (`groupBy`):
 - **Stage 1 (narrow):** scan → filter → broadcast-join. ~**16,000 tasks** (one per partition).
 - **Stage 2 (after shuffle):** read grouped data → `sum(amount)` . ~**200 shuffle partitions** (AQE may coalesce them).
1. **Tasks scheduled.** The **Task Scheduler** ships one task per partition. With 40 cores, ~40 run at once; the 16,000 Stage-1 tasks flow through in ~**400 waves**.
2. **Stage 1 runs.** Each task reads its 128 MB partition, filters, tags the region from the broadcast copy, and **writes shuffle files** to local disk, hashed by `region_name` .
3. **The shuffle (the costly part).** Between stages, data is **exchanged over the network**: each Stage-2 task pulls its region's pieces from every executor. This `Exchange` is what shows up in `df.explain()` .
4. **Stage 2 runs.** Each task sums `amount` for its regions.
5. **Write results.** Executors write the small output to **S3** (`curated/revenue_by_region/`). Had you called `collect()` , results would return to the **driver** — fine for tiny output, dangerous for large.
6. **Done.** Driver reports **SUCCEEDED** to the cluster manager; the orchestrator moves to the next step (crawl → load → quality checks).

3.5 Where Scala / the JVM fit

Spark's engine is written in **Scala and runs on the JVM** — driver and executors are JVM processes. **PySpark is a thin Python layer**: your Python driver talks to the JVM via **Py4J**, so **DataFrame/SQL operations run inside the JVM at Scala speed** (Python only builds the plan). The exception is **Python UDFs** — rows are serialized from the JVM to a **Python worker** on each executor and back, which is why UDFs are slower and interviewers say "prefer built-ins."

```

Python driver —Py4J→ JVM driver → JVM executors (DataFrame/SQL run here – fast)
                        |
                        └─(Python UDFs only)→ Python worker per executor
  
```

3.6 The numbers, made tangible

2 TB ÷ 128 MB ≈ **16,000 partitions/tasks** · 10 executors × 4 cores = **40 parallel tasks** → ~**400 waves** · join is **broadcast** (no big shuffle) · shuffle happens at `groupBy` · output is tiny (one row per region).

3.7 What can go wrong (and the fixes)

- **Data skew:** one region has most orders → one Stage-2 task lags → whole stage waits. Fix: **AQE skew join** or **salting**.
- **Forgot `broadcast()`** : the join would shuffle the full **2 TB** — minutes become hours.
- **Executor OOM:** partitions too big / huge shuffle / `collect()` to driver → raise memory, add partitions, avoid `collect()` .
- **Small files:** many output partitions → `coalesce` before writing.

PART 4 — The data engineer's responsibilities, mapped to the architecture

Component	What the DE actually does
Ingestion	Set up CDC/streaming, handle schema drift, retries, backfills, idempotency
Lake	Design zones, partitioning, file formats, lifecycle/retention, compaction
Spark processing	Write PySpark/Spark SQL, tune shuffles/joins/skew, right-size clusters
Catalog	Crawlers or explicit DDL + partition projection; keep schemas current
Warehouse/serving	Model star schemas, load via COPY, tune DIST/SORT keys, MVs
Orchestration	Build DAGs with retries/alerts; event triggers; dependencies
Quality	Tests that fail the pipeline before bad data reaches BI
Governance/security	Least-privilege IAM, KMS encryption, Lake Formation, PII handling
Monitoring/cost	Alarms on failures/latency/cost; partitioning + Parquet to cut scan cost
IaC / CI-CD	Terraform/CloudFormation; Git + CI to deploy pipelines safely

PART 5 — Cross-cutting concerns (what makes it production-grade)

- **Orchestration:** Airflow/Step Functions run the whole DAG (ingest → Spark → crawl → load → quality → publish) with retries, backfills, and alerting; event-driven via EventBridge on new S3 files.
- **Data quality:** dbt tests / Great Expectations (null, unique, range, freshness) that **block** a bad load.
- **Governance & security:** IAM least privilege, KMS encryption at rest, Lake Formation for table/column/row access, PII masking, CloudTrail audit.
- **Monitoring & cost:** CloudWatch metrics/alarms, the **Spark UI** for job/stage/task timings and shuffle read/write, cost control via partitioning, Parquet, right-sized clusters, and auto-scaling.
- **Reliability:** idempotent writes (overwrite-by-partition), checkpointing for streaming, fault tolerance from Spark's DAG lineage (lost partitions are recomputed).

PART 6 — The one-paragraph interview answer

"A data platform moves data from sources through ingestion into a lake, processes it with Spark, catalogs it, and serves it to a warehouse and BI/ML — all orchestrated, governed, and monitored, and defined as code. When I submit a Spark job, `spark-submit` starts the **driver**, which asks the **cluster manager** for **executors** on **worker nodes**. The driver builds a **DAG** from my lazy transformations, Catalyst optimizes it, and when an **action** fires the **DAG Scheduler** splits the job into **stages** at shuffle boundaries; the **Task Scheduler** sends **one task per partition** to executors that run them in parallel. Narrow ops stay on the partition; wide ops like groupBy/join cause a **shuffle** across the network. Spark's engine is Scala/JVM, and PySpark builds the plan via Py4J so DataFrame ops run at JVM speed. For a 2 TB revenue job I'd broadcast the small dimension, watch for skew, coalesce before writing, and partition the output by date — then the orchestrator catalogs it, loads the warehouse, runs the quality checks, and the dashboard refreshes."